

Referencia de Developer do ATC

Referencia de developer do ATC

Esta referencia complementa o Guia de Developer. O foco e pratico: como contribuir para o Advanced Trigonometry Calculator 2.1.7 sem introduzir regressoes.

Estrutura principal do projeto

- Advanced Trigonometry Calculator/: codigo C++ e ficheiros Visual Studio.
- tests/: regressao PowerShell, cobertura isolada e stress de memoria.
- docs/: documentacao Markdown, PDF e DOCX.
- tools/docs/: geradores de documentacao.

Expectativas de codigo

- Manter nomes publicos de comandos estaveis salvo quando existir plano de compatibilidade.
- Preferir helpers existentes de parser, alocao e formatacao.
- Manter comentarios de codigo em ingles.
- Evitar comportamento escondido que nao possa ser testado ou documentado.
- Preservar o objetivo de compatibilidade Windows XP ate Windows 11 onde o codigo atual o suporta.

Adicionar um novo comando

1. Localizar o handler mais proximo em `commands.cpp`, `main_processor.cpp` ou `main_aux_processor.cpp`. 2. Seguir o estilo atual de deteccao de comandos. 3. Manter prompts e outputs estaveis se forem testados. 4. Adicionar cobertura em `tests/run-atc-regression.ps1` quando possivel. 5. Documentar o comando no guia e na matriz de cobertura.

Adicionar uma funcao matematica

1. Confirmar se pertence a `trigonometry.cpp`, `hyperbolic.cpp`, `logarithmic.cpp`, `statistics.cpp`, `digital_signal_processing.cpp` ou a outro modulo existente. 2. Considerar caminhos `double` e `Boost mp_float`. 3. Confirmar comportamento de modo angular se for trigonometrica. 4. Adicionar vocabulario de autocomplete se a funcao deve ser sugerida. 5. Adicionar testes de regressao com output estavel.

Adicionar um modulo guiado

1. Manter o fluxo interativo deterministico sempre que pratico. 2. Evitar abrir janelas externas durante testes automaticos. 3. Adicionar pelo menos um smoke test seguro. 4. Documentar lacunas manuais se a automacao completa nao for pratica.

Adicionar um teste de regressao

Usar `tests/run-atc-regression.ps1` para verificacoes normais de comando/output. Os testes devem ser:

- deterministicos;
- independentes de caminhos absolutos locais;
- seguros com input redirecionado;
- isolados das settings do utilizador, ou com backup/restore pelo runner.

Nao atualizar a contagem documentada de testes sem executar a suite.

Documentar uma funcionalidade

Quando uma funcionalidade muda:

1. atualizar primeiro a referencia em ingles; 2. rever a versao portuguesa correspondente; 3. atualizar `tests/ATC_USER_GUIDE_COVERAGE.md`; 4. regenerar PDFs/DOCX quando relevante.

Riscos de regressao no parser e solver

Alteracoes no parser e solver devem ser testadas com:

- aritmetica direta;
- multiplicacao implicita;
- variaveis;
- valores complexos;
- simplificacao de polinomios;
- `solve equation(...)`;
- `solver(...)`;
- modo de alta precisao quando relevante.

Evitar resolver um caso especial contornando o fluxo normal, salvo se esse comportamento for deliberadamente documentado e testado.

Compatibilidade Windows

O ATC contem comportamento de consola para ambientes Windows antigos e novos. Ter cuidado com:

- comportamento especifico do Windows Terminal;
- output ANSI/VT;
- tamanho e posicao da janela;
- comandos que abrem novas instancias, abas, ficheiros ou pastas.

Estabilidade de output

Os testes automaticos normalmente validam padroes de output. Antes de alterar texto visivel, considerar se ele aparece em:

- documentacao de utilizador;
- notas de release;
- testes de regressao;
- relatorios exportados.

Checklist antes de commit/release

- Fazer build Release x64.
- Fazer build Release x86 quando houver alteracoes sensiveis a compatibilidade.
- Executar a suite de regressao.
- Executar cobertura isolada se a alteracao tocar comportamento interativo ou source-level.
- Executar stress de memoria para alteracoes com muitas alocoes.
- Atualizar documentacao e notas de release.

- Validar links Markdown e regenerar PDFs/DOCX quando necessario.